CS6400-A Project Proposal (Group 18)

Wei-Po Lin
wlin401@gatech.edu

Yu-An Chen
ychen3606@gatech.edu

You-Hsiang Tsai
ytsai309@gatech.edu

Siang-Yu Cheng
scheng371@gatech.edu

1 PROPOSED APPROACH

1.1 Motivation

Retrieval-Augmented Generation (RAG) is an architecture for optimizing the performance of large language models by connecting it with external knowledge bases outside of its training data sources[1, 5]. It allows LLMs to answer questions in various contexts and domains without the need to do expensive retraining of models, providing a cost-effective alternative to improving LLM output. RAGs filter large corpus to a small set of passages that are likely to contain the information needed to answer the query in question, then feed it to the LLM. We do this because it is often infeasible to feed an entire database as it will probably exceed the model's context window, and processing long contexts will also increase inference latency and computational cost. Therefore we choose to preprocess a relevant extract instead using RAG.

A key component of a RAG system is its retrieval strategy. Sparse retrieval methods such as BM25 identify relevant passages using keyword overlaps, while dense retrieval methods represent queries and documents as embeddings and select passages based on semantic similarity. These approaches capture complementary signals, so a hybrid system can efficiently combine ranked results using methods like Reciprocal Rank Fusion (RRF)[3, 4]. However, this initial hybrid ranking may still be inaccurate, in which case a more rigorous but computationally expensive method like a cross-encoder reranker can provide more relevant passages [8]. A cross-encoder processes each query-document pair jointly and produces a relevance score, allowing it to make more fine-grained relevance judgments than dense retrieval, where queries and documents are encoded independently. However, this additional computation makes cross-encoder reranking more expensive.

Our project proposes a latency-aware adaptive reranking system for hybrid RAG retrieval. We first retrieve candidates using BM25 and dense retrieval then combine their rankings using RRF. Rather than applying the cross-encoding reranker to every query, we estimate the confidence of the initial hybrid reranking using signals such as agreement between the sparse and dense retrievers and separation between the top candidates. If the retrieval results differ past a threshold, the system will invoke the cross-encoder to rerank the candidate passages. If the initial hybrid reranker is confident, then the system skips the second reranking stage and directly returns the top RRF passages. The contribution of our project is therefore the adaptive policy that determines when reranking is necessary. We will evaluate whether this policy can preserve retrieval quality of always reranking while reducing reranking calls and query latency.

1.2 End-to-End Architecture

Documents will first be parsed into textual sections and divided into fixed-sized overlapping chunks. Each chunk will retain metadata containing its document identifier and chunk identifier so that all generated answers can include source provenance. The chunks will then be indexed via a sparse index using BM25 and a dense index using vector embeddings generated from a pretrained model and stored in a vector index. For each query, BM25 and dense retrievers will independently go through their respective indexes and return the top K candidates. We will then combine them using Reciprocal Rank Fusion (RRF), a rank-based method that combines results from different retrieval systems without requiring their raw relevance scores to be directly comparable. The resulting candidates will then pass through an adaptive reranking system based on:

- **Retriever agreement**: overlap between the top K chunks returned by BM25 and dense retrieval
- **Ranking confidence**: the score separation between the top candidates

Ideally, we want the sparse and dense retrievers to rank similar candidates highly and for the top fused candidates to have a clear score difference. This suggests that the initial hybrid ranking is confident enough and additional reranking may not be necessary. However, if BM25 and dense retrieval rank the same candidates very differently, or if the top fused candidates have very similar scores, then the ranking may be uncertain and a more rigorous reranking step could help. Our system will use these signals to decide when to rerank based on thresholds tuned using development data and then kept fixed for final evaluation. When reranking is triggered, a cross-encoder will score each query-candidate pair and reorder the candidates based on relevance. The query and final top candidates will then be provided to the LLM along with their provenance, and each answer will identify the source document and supporting text chunk.

2 IMPLEMENTATION PLAN

2.1 Components reused

- Python programming language
- BM25 implementation e.g. rank-bm25 [2]
- pretrained model for dense retrieval e.g. sentence-transformers/all-MiniLM-L6-v2 [9]
- similarity search e.g. FAISS [6]
- cross-encoder e.g. e.g. cross-encoder/ms-marco-MiniLM-L6-v2 [7]
- LLM for answer generation e.g. OpenAI API using a fixed ChatGPT model

2.2 Components implemented

- parsing, cleaning, and normalization
- chunk construction and metadata management
- sparse and dense indexing
- consistent output format for sparse and dense retrieval results
- Reciprocal Rank Fusion
- adaptive reranking decision logic
- provenance tracking
- prompt construction and retrieved-context input
- retrieval evaluation
- latency measurement
- experiment configuration

3 EVALUATION PLAN

3.1 Metrics of Success

We will evaluate the system along three dimensions: retrieval quality, QA accuracy, and system efficiency. Retrieval quality is the primary evaluation target. On QASPER, we will report Recall@K and MRR, following standard information retrieval evaluation practices [10].

- **Recall@K**: Measures whether the gold evidence is retrieved within the top K retrieved passages. For example, if a question has two gold evidence passages and the top five results contain one of them, the Recall@5 for that question is 50%. We will report Recall@1, Recall@5, and Recall@10 to measure retrieval performance at different ranking depths.
- **MRR (Mean Reciprocal Rank)**: Measures how highly the first relevant evidence is ranked. For each question, the reciprocal rank is $1/r$, where r is the rank of the first relevant retrieved passage. For example, if the first relevant passage is ranked first, the score is 1; if it is ranked fifth, the score is 0.2. MRR is the average reciprocal rank across all questions.

On the secondary benchmark, we will use the metrics provided or commonly used for that benchmark.

QA accuracy is a secondary metric. We will pass the final retrieved passages to the same fixed LLM and evaluate generated answers against the ground-truth answers using answer F1.

System efficiency will be evaluated using reranking rate, average retrieval latency, and the number of cross-encoder calls. Our main goal is to achieve retrieval quality close to the always-reranking system while reducing reranking frequency and retrieval latency.

3.2 Benchmarks

We will evaluate on two benchmarks.

Primary: QASPER. We will use QASPER as required by the project specification. Its gold evidence annotations will be used for retrieval evaluation.

Secondary: BEIR-SciFact. We plan to use SciFact from the BEIR benchmark as our secondary retrieval benchmark [10]. We will report Recall@10, MRR@10, and NDCG@10 using the relevance judgments provided by the benchmark.

3.3 Evaluation Set

For QASPER, we plan to evaluate approximately 1,000 questions sampled from the designated test set using a fixed random seed. The same questions will be used for all retrieval systems.

For SciFact, we plan to evaluate the full test set if computationally feasible; otherwise, we will use a fixed randomly sampled subset and report the sampling procedure.

The evaluation set will not be used to tune the adaptive reranking thresholds.

3.4 Systems Compared

We will compare the following five configurations:

(1) **BM25** — required sparse retrieval baseline.
(2) **Dense Retrieval** — required dense retrieval baseline.
(3) **Hybrid RRF** — BM25 and dense retrieval combined using RRF.
(4) **Always Rerank** — Hybrid RRF followed by cross-encoder reranking for every query.
(5) **Adaptive Rerank** — our proposed method, which invokes the cross-encoder only when the retrieval confidence is below the selected threshold.

This comparison allows us to measure the contribution of hybrid retrieval, cross-encoder reranking, and the adaptive reranking policy separately.

3.5 Threshold Tuning and Fair Comparison

The adaptive reranking thresholds will be tuned using development data based on retriever agreement and ranking confidence, and then fixed for final evaluation without using the test set. All systems will use the same corpus, preprocessing, chunking strategy, evaluation queries, candidate pool size, and final Top-K setting. For QA evaluation, we will use the same LLM, prompt, and generation settings across all systems. Latency will be measured under the same hardware and software environment.

“‘latex

4 TIMELINE AND DIVISION OF WORK

We plan to begin implementation on October 1 and complete the project by December 15. The project will be divided into five milestones.

- **Milestone 1 (Oct. 1–12): Setup and Baselines.** Prepare the QASPER and secondary benchmark datasets, implement document preprocessing and chunking, and establish the required BM25 and dense retrieval baselines.
- **Milestone 2 (Oct. 13–26): Hybrid Retrieval.** Implement RRF-based hybrid retrieval, integrate the retrieval pipeline, and establish the initial retrieval evaluation pipeline. By the milestone report deadline, we plan to have a working preprocessing pipeline, both required baselines, and preliminary retrieval results.
- **Milestone 3 (Oct. 27–Nov. 9): Adaptive Reranking.** Integrate the cross-encoder, implement the retriever-agreement and ranking-confidence signals, and develop the adaptive reranking policy. Tune the decision thresholds using development data.

- **Milestone 4 (Nov. 10–30): Full Evaluation.** Run the complete evaluation on QASPER and the secondary benchmark, measure retrieval quality, QA accuracy, reranking frequency, and latency, and conduct ablation and comparative experiments.
- **Milestone 5 (Dec. 1–15): Finalization.** Analyze experimental results, finalize the RAG generation pipeline and provenance reporting, prepare the final report and presentation, and perform final testing and debugging.

The work will be divided among four team members based on the major components of the system:

- **Member 1 – Data and Preprocessing:** Dataset preparation, document parsing, cleaning, chunking, metadata management, and provenance tracking.
- **Member 2 – Retrieval and Indexing:** BM25 and dense retrieval, embedding generation, vector indexing, candidate retrieval, and RRF implementation.
- **Member 3 – Reranking and Adaptive Policy:** Cross-encoder integration, retriever-agreement and confidence signals, adaptive reranking logic, threshold tuning, and the always-rerank baseline.
- **Member 4 – Evaluation and Generation:** Retrieval metrics, benchmark evaluation, QA generation, answer F1, latency measurement, experiment tracking, and result analysis.

All team members will participate in system integration, final experiments, debugging, and report preparation. Evaluation infrastructure will be developed alongside the retrieval components rather than being deferred until the final stage. “‘

REFERENCES

[1] Amazon Web Services. [n.d.]. What Is Retrieval-Augmented Generation (RAG)? https://aws.amazon.com/what-is/retrieval-augmented-generation/. Accessed: 2026-09-30.

[2] Dorian Brown. [n.d.]. *rank_bm25: A Collection of BM25 Algorithms in Python.* https://github.com/dorianbrown/rank_bm25 Accessed: 2026-09-30.

[3] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Büttcher. 2009. Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval.* ACM, 758–759. https://doi.org/10.1145/1571941.1572114

[4] Google Cloud. [n.d.]. About Hybrid Search. https://cloud.google.com/vertex-ai/docs/vector-search/about-hybrid-search. Accessed: 2026-09-30.

[5] IBM. [n.d.]. What Is Retrieval-Augmented Generation (RAG)? https://www.ibm.com/think/topics/retrieval-augmented-generation. Accessed: 2026-09-30.

[6] Meta AI Research. [n.d.]. FAISS: A Library for Efficient Similarity Search and Clustering of Dense Vectors. https://faiss.ai/. Accessed: 2026-09-30.

[7] Sentence Transformers. [n.d.]. cross-encoder/ms-marco-MiniLM-L6-v2. https://huggingface.co/cross-encoder/ms-marco-MiniLM-L6-v2. Hugging Face model card. Accessed: 2026-09-30.

[8] Sentence Transformers. [n.d.]. Retrieve & Re-Rank. https://www.sbert.net/examples/sentence_transformer/applications/retrieve_rerank/README.html. Accessed: 2026-09-30.

[9] Sentence Transformers. [n.d.]. sentence-transformers/all-MiniLM-L6-v2. https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2. Hugging Face model card. Accessed: 2026-09-30.

[10] Nandan Thakur, Nils Reimers, Andreas Rücklé, Abhishek Srivastava, and Iryna Gurevych. 2021. BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models. In *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track.*